Verdiepende Python Cursus: deel 1

Leerdoelen

- ☐ Pythonontwikkelomgeving opzetten in VSCode.
- ☐ Leren werken met virtuele omgevingen in Python.
- ☐ Basis van projectstructuur.

Veiligheid

We gaan programmeren, hier zijn geen veiligheidskwesties aan gebonden. Probeer alleen zoveel mogelijk zelfstandig te programmeren. Gebruik ChatGPT verstandig:

1. Gebruik het als uitlegmachine, niet als codegenerator
2. Gebruik het om eigen code te laten verbeteren, maar niet herschrijven
3. Gebruik het niet voor volledige oplossingen, snap je iets niet? Vraag het de docent.

Opdrachten: Werken met VSCode en professionele workflow

Deel 1: Installeer Python en VSCode-extensies.

1 **Installeer Python en VSCode-extensies.**
Controleer of Python is geïnstalleerd (`python --version`). Installeer in VSCode de extensies **Python** en **Pylance**.

Tips & Tricks: Conda verschijnt niet in de VSCode-terminal

- Zie je geen `(base)` in VSCode? Dan is conda niet geïnitialiseerd voor deze shell.
- Gebruik je PowerShell? Initialiseer conda via: `conda init powershell` (in Anaconda Prompt).
- Gebruik je Command Prompt (cmd)? Initialiseer conda via: `conda init cmd.exe`.
- Na initialisatie: sluit VSCode volledig af en open opnieuw.
- Open een *nieuwe* terminal in VSCode — oude terminals laden conda niet in.

Deel 2: Projectmap en eerste bestanden

1. Maak een nieuwe map aan, bijvoorbeeld `python_intro`, en open deze map in VSCode.
2. Maak in deze map (optioneel) een submap voor je scripts.
3. Maak een nieuw Pythonbestand `helloworld.py` in de projectmap met de volgende inhoud:

helloworld.py

```python
import numpy as np

print("Hello world")

a = np.array([1, 2, 3, 4])
print("Array a:", a)
print("Array a^2:", a**2)
```

1. Maak in dezelfde map een nieuw Jupyter Notebook `helloworld.ipynb`. Voeg in de eerste cel dezelfde code toe als in `helloworld.py`:

helloworld.ipynb – eerste cel

```python
import numpy as np

print("Hello world from notebook")

a = np.array([1, 2, 3, 4])
print("Array a:", a)
print("Array a^2:", a**2)
```

Deel 3: Virtuele omgeving aanmaken via Anaconda Prompt

Open de **Anaconda Prompt** en voer de onderstaande stappen uit:

Anaconda Prompt: virtuele omgeving en installatie

```
# Maak een nieuwe virtuele omgeving via Anaconda Prompt
(base) C:\Users\student> conda create -n python_cursus python=3.11
(base) C:\Users\student> conda activate python_cursus

# Installeer de benodigde pakketten
(python_cursus) C:\Users\student> pip install numpy ipykernel
```

Deel 4: Test `helloworld.py` in de virtuele omgeving

1. Zorg dat de omgeving `python_cursus` actief is in VSCode.
   Kies rechtsboven in VSCode bij “Interpreter” voor: `python_cursus`.
2. Controleer in de terminal (onderin VSCode) of de omgeving actief is:
   `conda activate python_cursus`
3. Navigeer in de terminal naar jouw projectmap, bijvoorbeeld:

VSCode Terminal: navigeren naar projectmap

```
(python_cursus) PS C:\Users\student> cd Documents\python_intro
```

1. Voer nu het script `helloworld.py` uit. Dit kan op twee manieren:
   - **Via de Run-knop** in VSCode (rechtsboven in het editor-venster).
   - **Via de terminal**:

VSCode Terminal: uitvoeren van het script

```
(python_cursus) PS C:\Users\student\python_intro> python helloworld.py
```

1. Controleer of de volgende output verschijnt:
   - De tekst `Hello world`
   - De NumPy-array en de gekwadrateerde waarden

Veelvoorkomende fouten en tips

Veelvoorkomende fouten

- Script wordt uitgevoerd met de verkeerde interpreter (bijv. `base`).
- Interpreter in VSCode staat niet op `python_cursus`, ondanks de Run-knop.
- Terminal in VSCode gebruikt PowerShell zonder geactiveerde omgeving.
- Pakketten (zoals NumPy) ontbreken omdat ze in de verkeerde omgeving zijn geïnstalleerd.
- Vekeerde map geopend.
- Terminal toont: `ModuleNotFoundError: No module named 'numpy'`.

Tips & tricks

- Controleer in VSCode rechtsboven: interpreter moet `python_cursus` zijn.
- Open een nieuwe terminal in VSCode → omgeving wordt meestal automatisch geactiveerd.
- Check actief Python-pad met: `where python`.
- Voordat je runt: controleer dat je in de juiste projectmap staat.
- Installeer pakketten altijd ná `conda activate python_cursus`.
- Bij foutmeldingen: terminal sluiten, opnieuw openen, interpreter opnieuw kiezen.

Deel 5: Test `helloworld.ipynb` in de virtuele omgeving

1 Zorg dat de omgeving `python_cursus` een Jupyter-kernel heeft (eenmalig nodig):

Anaconda Prompt: kernel installeren

```
(python_cursus) C:\Users\student> python -m ipykernel install --user ^
    --name python_cursus --display-name "python_cursus"
```

1 Open VSCode, en open daarin `helloworld.ipynb`.
2 Selecteer bovenin het notebook als kernel/interpreter: `python_cursus`.
3 Run alle cellen van het notebook.
4 Controleer of de uitvoer overeenkomt met de uitvoer van `helloworld.py` (tekst en NumPy-array).

Veelvoorkomende fouten en tips

Veelvoorkomende fouten

- Notebook gebruikt een verkeerde kernel (meest voorkomend).
- De omgeving verschijnt niet als kernel omdat `ipykernel` ontbreekt.
- VSCode valt terug op de `base`-omgeving.
- Cell blijft hangen op “busy” door verkeerde kernelselectie.
- Packages ontbreken doordat ze in de verkeerde omgeving zijn geïnstalleerd.

Tips & tricks

- Controleer rechtsboven in VSCode: staat de kernel op `python_cursus`?
- Herstart VSCode als de kernel niet zichtbaar is.
- Controleer welke Python gebruikt wordt met `where python`.
- Voer `pip list` uit in een cell om te checken dat NumPy in de juiste omgeving zit.
- Los notebookproblemen op via: *Kernel → Restart kernel*.

Appendix 1:

Anaconda Prompt: handige standaardcommando's

```
# Toon alle conda-omgevingen
(base) C:\Users\student> conda env list

# Nieuwe omgeving aanmaken
(base) C:\Users\student> conda create -n python_cursus python=3.11

# Een omgeving activeren / deactiveren
(base) C:\Users\student> conda activate python_cursus
(python_cursus) C:\Users\student> conda deactivate

# Installeer pakketten in de actieve omgeving
(python_cursus) C:\Users\student> pip install numpy matplotlib scipy

# Toon alle geïnstalleerde pakketten (conda)
(python_cursus) C:\Users\student> conda list

# Update een pakket
(python_cursus) C:\Users\student> conda install --upgrade numpy

# Verwijder een pakket
(python_cursus) C:\Users\student> conda uninstall numpy

# Verwijder een volledige omgeving
(base) C:\Users\student> conda remove -n python_cursus --all
```

Appendix 2:

Handige VSCode-extensies voor Python

- **Python** — officiële extensie voor Python-ondersteuning.
- **Pylance** — snelle en slimme taalserver (typechecking, foutdetectie).
- **Jupyter** — notebooks openen en uitvoeren in VSCode.
- **Jupyter Keymap** — handige notebook-sneltoetsen (zoals in JupyterLab).
- **Jupyter Cell Tags** — tags voor complexe notebooks.
- **GitLens** — versiebeheer, commits bekijken, file history.
- **EditorConfig** — consistente code-stijl binnen projecten.
- **Indent Rainbow** — kleurcodering van inspringing (minder fouten).
- **Better Comments** — TODO/FIXME/NOTE visueel markeren.
- **Material Icon Theme** — duidelijke iconen voor Python-projectstructuur.

Verdiepende Python Cursus: deel 2

Leerdoelen

- ☐ Herkennen van verschillende data-structuren.
- ☐ Leren werken en toepassen van vector-operaties in Python.
- ☐ Functies, doc-strings, opties & plotten.

Veiligheid

We gaan programmeren, hier zijn geen veiligheidskwesties aan gebonden. Probeer alleen zoveel mogelijk zelfstandig te programmeren. Gebruik ChatGPT verstandig:

1. Gebruik het als uitlegmachine, niet als codegenerator
2. Gebruik het om eigen code te laten verbeteren, maar niet herschrijven
3. Gebruik het niet voor volledige oplossingen, snap je iets niet? Vraag het de docent.

Opdrachten:

Maak een nieuw Jupyter Notebook aan met de naam `week2.ipynb`. Voer alle onderstaande onderdelen uit als losse cellen in dit notebook.

Deel 1: Datastructuren

Q1

Maak in `week2.ipynb` een eerste codecel waarin je voorbeelden maakt van (Link):

a) een lijst;
b) een array;
c) een dictionary.

Oplossing:

Cel 1 – voorbeelden datastructuren

```
# Voorbeelden van datastructuren
lijst = [1, 2, 3, 4]
dictionary = {"a": 1, "b": 2, "c": 3}
array = np.array([1,2,3,4])

print("Lijst:", lijst)
print("Dictionary:", dictionary)
print("Array:", array)
```

Q2

a) Maak een tweede codecel waarin je in commentaar (`# ...`) kort noteert wanneer je welke datastructuur zou gebruiken (bijv. lijst vs. dictionary).
b) Bereken het kwadraat van de waardes in de `lijst`.
c) Bereken het kwadraat van de waardes in de NumPy-`array`.
d) Wat gebeurt er als je een lijst vermenigvuldigt met een integer?
e) Waarvoor zou je een `dictionary` kunnen gebruiken?

Oplossing:

a) Je gebruikt een lijst om dingen in op te slaan (net als een boodschappenlijstje). Dit kunnen dus **strings**, **integers**, etc. zijn. Een array (NumPy-array) is bedoeld voor numerieke berekeningen; de elementen zijn typisch **floats** en **integers** en je kunt er efficiënt wiskundige operaties mee uitvoeren. Een dictionary is een soort bibliotheek: je haalt elementen op aan de hand van een *key*. Dit wordt minder vaak gebruikt binnen de basis-natuurkunde, maar is handig voor gestructureerde data.

b) **Cel – kwadraat van de lijst**

```
lijst = [1, 2, 3, 4]
lijst_kwadraat = [i**2 for i in lijst]

print("Lijst:", lijst)
print("Lijst in het kwadraat:", lijst_kwadraat)
```

c) **Cel – kwadraat van de array**

```
import numpy as np

array = np.array([1, 2, 3, 4])
array_kwadraat = array**2  # vectoriële operatie

print("Array:", array)
print("Array in het kwadraat:", array_kwadraat)
```

d) Als je een integer probeert te vermenigvuldigen met een lijst, dan wordt de lijst herhaald in plaats van elementgewijs vermenigvuldigd.
   Bijvoorbeeld: `3 * [1, 2, 3]` resulteert in `[1, 2, 3, 1, 2, 3, 1, 2, 3]`.
   Python voert dus geen wiskundige vermenigvuldiging uit per element — het ziet de operatie als “lijst drie keer achter elkaar plakken”.
e) Een dictionary kun je bijvoorbeeld gebruiken om:
   - meetdata per sample-ID op te slaan, bijv. `metingen = {"sample1": [1.2, 1.3], "sample2": [0.9, 1.0]}`
   - fysische constanten te bewaren, bijv. `constanten = {"g": 9.81, "c": 3.00e8}`
   - experimentele instellingen vast te leggen, bijv. `config = {"temperatuur": 293, "druk": 1.0}`.

Deel 2: Vector-operaties met `numpy`

Q3

Maak een nieuwe codecel waarin je `numpy` importeert en twee vectoren definieert.

Oplossing:

Cel 3 – basis vector-operaties

```
import numpy as np

v1 = np.array([1, 2, 3, 4, 5])
v2 = np.array([5, 4, 3, 2, 1])

print("v1 =", v1)
print("v2 =", v2)
```

Q4

Bereken het product (elementsgewijs) van de vectoren.

Oplossing:

Cel – elementgewijs product van twee vectoren

```
import numpy as np

v1 = np.array([1, 2, 3, 4, 5])
v2 = np.array([5, 4, 3, 2, 1])

# Elementgewijs product
element_product = v1 * v2

print("v1 * v2 =", element_product)
```

Q5

Bereken het dotproduct tussen de vectoren.

Oplossing:

Cel – dotproduct van twee vectoren

```
import numpy as np

v1 = np.array([1, 2, 3, 4, 5])
v2 = np.array([5, 4, 3, 2, 1])

# Dotproduct (inwendig product)
dot = np.dot(v1, v2)

print("dot(v1, v2) =", dot)
```

Q6

Bereken het dotproduct tussen dezelfde twee vectoren, maar dit keer zonder `numpy` — gebruik gewone Python-lijsten en een `for`-loop.

Oplossing:

Cel – dotproduct met lijsten en for-loops

```
# Dotproduct berekenen met lijsten en een for-loop

v1 = [1, 2, 3, 4, 5]
v2 = [5, 4, 3, 2, 1]

dot = 0
for i in range(len(v1)):
    dot += v1[i] * v2[i]

print("Dotproduct (met for-loop) =", dot)
```

Q7

Maak een extra codecel waarin je de norm (lengte) van een vector berekent, en bereken ook de hoek tussen twee vectoren. Hint! De norm van de vector wordt veel gebruikt en heeft binnen numpy een standaard functie. Link. Verder wordt de hoek tussen twee vectoren weergeven als $\cos(\theta) = \frac{\vec{v_1} \cdot \vec{v_2}}{|v_1||v_2|} =$

Oplossing:

Cel 3 – norm en hoek tussen twee vectoren

```
# Norm (lengte) van beide vectoren
norm_v1 = np.linalg.norm(v1)
norm_v2 = np.linalg.norm(v2)

print("||v1|| =", norm_v1)
print("||v2|| =", norm_v2)

# Hoek tussen de twee vectoren (in radialen en graden)
cos_theta = np.dot(v1, v2) / (norm_v1 * norm_v2)
theta_rad = np.arccos(cos_theta)
theta_deg = np.degrees(theta_rad)

print("cos(theta) =", cos_theta)
print("theta (rad) =", theta_rad)
print("theta (deg) =", theta_deg)
```

Deel 3: Functies, docstrings & plotten

Docstrings. Een docstring is een korte beschrijving die je *in* een functie zet om uit te leggen wat deze doet, welke parameters de functie verwacht en wat de functie teruggeeft. In Python plaats je een docstring direct onder de functiedefinitie tussen drie dubbele aanhalingstekens: `""" ... """`. Tools zoals VSCode kunnen deze docstrings automatisch tonen wanneer je met de muis over een functie hovert.

VSCode-extension. De extensie die ondersteuning biedt voor docstrings en functie-informatie is **Pylance**. Deze zorgt voor slimme hints, type-informatie en toont automatisch de docstring van jouw functies én van Python-bibliotheken.

Auto-docstring. Voor het automatisch genereren van docstrings bestaat de extensie **autoDocstring**. Wanneer je in een functie `"""` typt en op `Enter` drukt, vult deze extensie automatisch een docstring-sjabloon in met secties voor beschrijving, parameters en returnwaarden. Dit helpt je om consistente, duidelijke documentatie voor al je functies te schrijven.

Q8

Maak een nieuwe codecel waarin je een functie definieert met een docstring die een vector elementgewijs tot een bepaalde macht verheft.

Oplossing:

Cel – functie met docstring

```python
import numpy as np

def macht(vector, power=2):
    """
    Verhoogt elk element van een vector tot de gegeven macht.

    Parameters
    ----------
    vector : array-like
        Invoervector (lijst of numpy-array).
    power : int of float, optioneel
        De macht waarmee elk element wordt verheven (standaard: 2).

    Returns
    -------
    numpy.ndarray
        Vector waarbij elk element is verheven tot 'power'.
    """
    vector = np.array(vector)
    return vector**power

print(macht([1, 2, 3]))           # gebruikt standaard power=2
print(macht([1, 2, 3], power=3))  # gebruiker kiest zelf de macht
```

Q9

Ik wil graag dat deze functie een standaardwaarde van 2 meegeeft aan de macht; als gebruiker heb ik dan een optie om deze aan te passen. Leg uit hoe dit werkt.

Oplossing: Dit gebeurt via de parameterdefinitie `power=2`. Als de gebruiker niets invult, wordt `power` automatisch 2. Wanneer de gebruiker zelf een waarde meegeeft, bijvoorbeeld `power=5`, dan overschrijft dit de standaardwaarde en wordt de vector tot die macht verheven.

Q10

Waar ben je in het verleden nog meer `opties` (default parameters) tegengekomen?

Oplossing: Opties (standaardwaarden) zag je eerder bij functies zoals:

- `print(..., sep=" ", end="")` — `sep` en `end` hebben standaardwaarden.

- `np.linspace(start, stop, num=50)` — `num` heeft een standaardwaarde.
- `plt.plot(x, y, label=None)` — veel plotopties zijn optioneel.

Q11

Maak een nieuwe codecel waarin je een eenvoudige plot (Link) maakt van de functie $f(x) = x^2$ op het interval $[-3, 3]$.

Oplossing:

Cel 6 – plot van f(x) = x2**

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-3, 3, 200)
y = x**2

plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("f(x) = x^2")
plt.grid(True)
plt.show()
```

Q12

Maak een codecel waarin je een plot maakt met twee y-assen: op de linker-as $f(x) = x^2$ en op de rechter-as $g(x) = \sin(x)$, beide uitgezet op hetzelfde x-interval.

Oplossing:

Cel – plot met twee y-assen

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-3, 3, 400)
y1 = x**2          # linker y-as
y2 = np.sin(x)     # rechter y-as

fig, ax1 = plt.subplots()

# Linker y-as
ax1.plot(x, y1, color="blue")
ax1.set_xlabel("x")
ax1.set_ylabel("f(x) = x^2", color="blue")
ax1.tick_params(axis="y", labelcolor="blue")

# Rechter y-as
ax2 = ax1.twinx()
ax2.plot(x, y2, color="red")
ax2.set_ylabel("g(x) = sin(x)", color="red")
ax2.tick_params(axis="y", labelcolor="red")

plt.title("Plot met twee y-assen")
plt.grid(True)
plt.show()
```

Q13

Maak een codecel waarin je een **log–log plot** maakt van $f(x) = x^3$ op het interval $[0.1, 10]$.

Oplossing:

Cel – log–log plot van f(x) = x3**

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0.1, 10, 400)   # kleinste waarde > 0 voor log-schaal
y = x**3

plt.loglog(x, y)
plt.xlabel("x (log-schaal)")
plt.ylabel("f(x) = x^3 (log-schaal)")
plt.grid(True, which="both")
plt.show()
```

Q14

Maak een codecel waarin je twee functies in **subplots** (Link) plaatst: boven $f(x) = \sin(x)$ en onder $g(x) = \cos(x)$, beide op het interval $[0, 2\pi]$.

Oplossing:

Cel – subplots met sin en cos

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2*np.pi, 400)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(6, 6), sharex=True)

# Bovenste subplot
ax1.plot(x, np.sin(x), color="blue")
ax1.set_ylabel("sin(x)")
ax1.grid(True)

# Onderste subplot
ax2.plot(x, np.cos(x), color="red")
ax2.set_ylabel("cos(x)")
ax2.set_xlabel("x")
ax2.grid(True)

plt.tight_layout()
plt.show()
```

Appendix 1: data-types

Python werkt met verschillende soorten gegevens, zogenaamde *datatypes*. De belangrijkste basistypen zijn:

`int`
: Een geheel getal (integer), bijvoorbeeld `3`, `-17`, `42`. Wordt gebruikt voor tellingen, indexen en numerieke berekeningen zonder decimalen.

`float`
: Een decimaal getal (floating point), bijvoorbeeld `3.14`, `0.001`. Dit datatype gebruik je voor fysische grootheden, metingen en elke numerieke berekening waarin decimalen voorkomen.

`str`
: Een tekststring (string), bijvoorbeeld `"Hallo"`, `"3.14"`. Strings gebruik je voor namen, labels, paden, bestandsnamen en alle niet-numerieke data.

`bool`
: Een booleaanse waarde: `True` of `False`. Wordt gebruikt in logica, voorwaarden (`if`) en vergelijkingen.

`list`
: Een geordende verzameling waarden, bijvoorbeeld `[1, 2, 3]` of `["A", "B"]`. Kan verschillende datatypes bevatten en is flexibel, maar minder efficiënt voor wiskundige operaties.

`numpy.ndarray`
: Een numerieke array van vaste grootte en datatype. Dit is het standaard gereedschap voor vector- en matrixberekeningen in de natuurkunde.

Python bepaalt het datatype automatisch, maar je kunt het altijd opvragen met `type(…)`.

Q15

Maak een codecel waarin je voor verschillende variabelen het datatype opvraagt met behulp van `type()`. Gebruik ten minste een `int`, `float`, `str`, `bool`, `list` en een `numpy.ndarray`.

Oplossing:

Voorbeeld – datatype opvragen met `type()`

```python
# Voorbeelden van datatypes en hoe je ze opvraagt

a = 3
b = 3.14
c = "Hallo"
d = True
e = [1, 2, 3]
import numpy as np
f = np.array([1, 2, 3])

print(type(a))   # <class 'int'>
print(type(b))   # <class 'float'>
print(type(c))   # <class 'str'>
print(type(d))   # <class 'bool'>
print(type(e))   # <class 'list'>
print(type(f))   # <class 'numpy.ndarray'>
```

Verdiepende Python Cursus: Optie 1

Leerdoelen

- ☐ CSV-bestanden inlezen en bewerken.
- ☐ Basis datacleaning: NaNs verwijderen, duplicaten verwijderen en (Boolean) filteren.
- ☐ Lineaire en kubische interpolaties toepassen op een simpele dataset.

Veiligheid

We gaan programmeren, hier zijn geen veiligheidskwesties aan gebonden. Probeer alleen zoveel mogelijk zelfstandig te programmeren. Gebruik ChatGPT verstandig:

1. Gebruik het als uitlegmachine, niet als codegenerator
2. Gebruik het om eigen code te laten verbeteren, maar niet herschrijven
3. Gebruik het niet voor volledige oplossingen, snap je iets niet? Vraag het de docent.

Opdrachten: Inlezen en verwerken van meetdata

Q16

Maak een nieuwe codecel waarin je het CSV-bestand `space_jump_synthetic.csv` inleest. Het bestand bevat een header van 4 regels en een footer van 2 regels. Lees de data in met `skiprows` en `skipfooter`, en sla de kolommen tijd, hoogte, snelheid en Mach-getal op in aparte variabelen.

Oplossing:

Cel – inlezen met `skiprows` en `skipfooter`

```
import pandas as pd

# CSV-bestand inlezen met header + footer die wegnemen worden
df = pd.read_csv(
    "space_jump_synthetic.csv",
    sep=";",
    skiprows=4,      # eerste 4 regels zijn header
    skipfooter=2,    # laatste 2 regels zijn footer
    engine="python"  # verplicht als skipfooter wordt gebruikt
)

# Kolommen opslaan in aparte variabelen
t = df["t [s]"]
h = df["h [m]"]
v = df["v [m/s]"]
mach = df["Mach [-]:"]

print(t.head())
print(h.head())
print(v.head())
print(mach.head())
```

Q17

Maak een codecel waarin je twee **subplots** maakt: boven de hoogte $h(t)$ en onder de snelheid $v(t)$, beide als functie van de tijd. Gebruik de ingelezen gegevens uit `space_jump_synthetic.csv`.

Oplossing:

Cel – subplots van hoogte en snelheid

```
import numpy as np
import matplotlib.pyplot as plt

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 7), sharex=True)

# Hoogte subplot
ax1.plot(t, h, color="blue")
ax1.set_ylabel("Hoogte h(t) [m]")
ax1.set_title("Space Jump: Hoogte en snelheid als functie van de tijd")
ax1.grid(True)

# Snelheid subplot
ax2.plot(t, v, color="red")
ax2.set_ylabel("Snelheid v(t) [m/s]")
ax2.set_xlabel("Tijd t [s]")
ax2.grid(True)

plt.tight_layout()
plt.show()
```

Q18

Maak een codecel waarin je de **potentiële energie** $E_p = mgh(t)$ en de **kinetische energie** $E_k = \frac{1}{2}mv(t)^2$ berekent (neem $m = 90\,\mathrm{kg}$, $g = 9.81\,\mathrm{m/s^2}$). Maak vervolgens één figuur waarin beide energieën als functie van de tijd worden geplot.

Oplossing:

Cel – potentiële en kinetische energie in één figuur

```
# Parameters
m = 90.0            # massa [kg]
g = 9.81            # zwaartekrachtsversnelling [m/s^2]

# Energieën
E_p = m * g * h          # potentiële energie [J]
E_k = 0.5 * m * v**2     # kinetische energie [J]

# Plot
plt.figure(figsize=(8, 5))
plt.plot(t, E_p, label="Potentiële energie $E_p$", color="blue")
plt.plot(t, E_k, label="Kinetische energie $E_k$", color="red")

plt.xlabel("Tijd t [s]")
plt.ylabel("Energie [J]")
plt.title("Potentiële en kinetische energie tijdens de sprong")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

Data filtering/cleaning

Q19

Maak een nieuwe codecel waarin je het CSV-bestand `space_jump_synthetic_nosiy.csv` inleest. Het bestand bevat een header van 4 regels en een footer van 2 regels. Lees de data in met `skiprows` en `skipfooter`, en sla de kolommen tijd, hoogte, snelheid en Mach-getal op in aparte variabelen. Het bestand bevat verder verschillende NaN's, duplicate rijen en "–1" waardes (soms wordt foute data niet als NaN genoteerd, maar als aparte waarde).

Q20

Filter de data, gebruik hiervoor de volgende code', maak vervolgens een zelfde soort grafiek van de data.

Cel – filter NaN

```
#Vervang -1 waardes door NaN waardes.
df = df.drop_duplicates()
df = df.replace(-1, np.nan)

#Filter hele dataframe op NaN waardes.
df_clean = df.dropna()

#Filter op basis van kolom.
df_clean = df.dropna(subset=["kolomnaam"])

#Check waar de NaNs zitten.
df_nan = df[df.isna().any(axis=1)]
```

Oplossing:

Cel – subplots van hoogte en snelheid

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

df = df.drop_duplicates()
df = df.replace(-1, np.nan)
df_clean = df.dropna()

t_clean = df_clean["t [s]"]
h_clean = df_clean["h [m]"]
v_clean = df_clean["v [m/s]"]
mach_clean = df_clean["Mach [-]:"]

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 7), sharex=True)

# Hoogte subplot
ax1.plot(t_clean, h_clean, color="blue")
ax1.set_ylabel("Hoogte h(t) [m]")
ax1.set_title("Space Jump: Hoogte en snelheid als functie van de tijd")
ax1.grid(True)

# Snelheid subplot
ax2.plot(t_clean, v_clean, color="red")
ax2.set_ylabel("Snelheid v(t) [m/s]")
ax2.set_xlabel("Tijd t [s]")
ax2.grid(True)
```

```
plt.tight_layout()
plt.show()
```

Q21

Zoals misschien is opgevallen, staat er naast NaN-waardes en duplicaten ook *outliers* in. *Outliers* verwijderen is iets moeilijker, we moeten hiervoor namelijk zeker weten of dit niet een meting was. Vaak kan je dit door inspectie doen óf door slimmer filtering te doen. Filter de *outliers*, Zie onderstaand voorbeeld ter uitleg.

Cel – filter on values

```
#Verwijder rijen (selecteer subset) op basis van criterium dat hoogte moet zijn
dan 40 km.
df = df[df["h [m]" < 40000]
```

Oplossing:

Cel – subplots van hoogte en snelheid

```
df_clean = df_clean[(df_clean["h [m]"] < 40000) & (df_clean["h [m]"] > 1000)]
#Kan ook een nieuwe variabele df_clean2 zijn.
t_clean = df_clean["t [s]"]
h_clean = df_clean["h [m]"]
v_clean = df_clean["v [m/s]"]
mach_clean = df_clean["Mach [-]:"]

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 7), sharex=True)

# Hoogte subplot
ax1.plot(t_clean, h_clean, color="blue")
ax1.set_ylabel("Hoogte h(t) [m]")
ax1.set_title("Space Jump: Hoogte en snelheid als functie van de tijd")
ax1.grid(True)

# Snelheid subplot
ax2.plot(t_clean, v_clean, color="red")
ax2.set_ylabel("Snelheid v(t) [m/s]")
ax2.set_xlabel("Tijd t [s]")
ax2.grid(True)

plt.tight_layout()
plt.show()
```

Interpolation

De hoogte van de sprong wordt slechts op discrete tijdstippen gemeten. Om de hoogte op een tussentijdse tijd t te schatten kan interpolatie worden gebruikt, waarbij wordt aangenomen dat de hoogte tussen twee opeenvolgende meetpunten continu varieert. In de eenvoudigste benadering wordt lineaire interpolatie toegepast tussen (t_1, h_1) en (t_2, h_2):

$$h(t) = h_1 + \frac{t - t_1}{t_2 - t_1}(h_2 - h_1). \tag{1}$$

Hiermee kan bijvoorbeeld de hoogte op een specifiek tijdstip tijdens zijn val worden benaderd, ook wanneer dit tijdstip niet exact gemeten is. Zie onderstaand voorbeeld:

Cel – Interpolatie voorbeeld

```
from scipy.interpolate import interp1d

v_interpolate = interp1d(t_clean, v_clean, kind="linear")

v_interpolate = f(2.5)
```

Q22

Maak een schatting van de hoogte en de snelheid op $t = 210$ seconden. Gebruik daarvoor een lineaire én een cubische interpolatie.

Oplossing:

Cel - Inschatting hoogte en snelheid

```
from scipy.interpolate import interp1d

h_interpolate_linear = interp1d(t_clean, h_clean, kind="linear")
h_interpolate_cubic = interp1d(t_clean, h_clean, kind="cubic")

print("h(210) linear: ", h_interpolate_linear(210))
print("h(210) cubic: ", h_interpolate_cubic(210))

v_interpolate_linear = interp1d(t_clean, v_clean, kind="linear")
v_interpolate_cubic = interp1d(t_clean, v_clean, kind="cubic")

print("v(210) linear: ", v_interpolate_linear(210))
print("v(210) cubic: ", v_interpolate_cubic(210))
```

Verdiepende Python Cursus: optie 2

Leerdoelen

- ☐ Niet-lineaire fits uitvoeren met `scipy.optimize.curve_fit`.
- ☐ Fysische parameters uit een fit afronden en interpreteren.
- ☐ Onzekerheden bepalen uit de covariantiematrix.

Veiligheid

We gaan programmeren, hier zijn geen veiligheidskwesties aan gebonden. Probeer alleen zoveel mogelijk zelfstandig te programmeren. Gebruik ChatGPT verstandig:

1. Gebruik het als uitlegmachine, niet als codegenerator
2. Gebruik het om eigen code te laten verbeteren, maar niet herschrijven
3. Gebruik het niet voor volledige oplossingen, snap je iets niet? Vraag het de docent.

Opdrachten:

Model

Het uitvoeren van een (niet-)lineaire regressie op een dataset vereist eerst een fysisch of empirisch model dat het verwachte gedrag van de data beschrijft. Dit model bevat één of meerdere parameters die niet direct bekend zijn en uit de data moeten worden bepaald. Het doel van curve fitting is om deze parameters zo te kiezen dat het verschil tussen het model en de gemeten data zo klein mogelijk wordt, meestal in de zin van een kleinste-kwadratencriterium. In Python kan dit proces efficiënt worden uitgevoerd met de functie `curve_fit` uit de `SciPy`-bibliotheek. In deze cursus gebruiken we als model een exponentiële afname, die bijvoorbeeld optreedt bij ontlaadprocessen of relaxatieverschijnselen.

$$N(t) = N_0 e^{-t/\tau},$$

Curve fitting

De functie `curve_fit` bepaalt de modelparameters door het verschil tussen het model en de gemeten data te minimaliseren volgens het gewogen kleinste-kwadratenprincipe. Concreet wordt de som van de kwadraten van de residuen geminimaliseerd (gecorrigeerd voor het aantal vrije datapunten):

$$\tilde{\chi}^2 = \frac{1}{N-d} \sum_i \frac{(y_i - f(t_i, \theta))^2}{\sigma_i^2},$$

waar y_i de gemeten waarden zijn, $f(t_i, \theta)$ het model, θ de te bepalen parameters en σ_i de onzekerheid op het betreffende datapunt.

De belangrijkste invoer voor `curve_fit` is een model-functie $f(t, \theta)$ (in ons geval dus een exponentieel verband), een array met de onafhankelijke variabele (bijvoorbeeld tijd t) en een array met de gemeten waarden $N(t)$. Optioneel kunnen ook meetfouten, beginwaarden en grenzen voor de parameters worden meegegeven.

De uitvoer bestaat uit de beste schatting van de modelparameters en de covariantiematrix. Uit de covariantiematrix wordt de onzekerheden op deze parameters bepaald.

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

def decay(t, y0, tau):
    return y0 * np.exp(-t/tau)

# gesimuleerde data met ruis
t = np.linspace(0, 10, 200)
y = decay(t, 5.0, 2.0) + 0.2*np.random.randn(len(t))

popt, pcov = curve_fit(decay, t, y)
y0_fit, tau_fit = popt
y0_err, tau_err = np.sqrt(np.diag(pcov))

plt.plot(t, y, ".", label="data")
plt.plot(t, decay(t, *popt), "-", label="fit")
plt.xlabel("t (s)")
plt.ylabel("y(t)")
plt.legend()
plt.grid()
plt.show()
```

```
print("y0 =", y0_fit, "+/-", y0_err)
print("tau =", tau_fit, "+/-", tau_err)
```

Korte opdracht

Voer een fit uit voor een curve die jij krijgt of zelf genereert. Rapporteer de optimale parameters en hun onzekerheden. Rapporteer ook de fysische parameters.